The autodiff cookbook: JVPs and VJPs

Print to PDF

Contents

- Setup: a running example
- Jacobians and Hessians using `jacfwd` and `jacrev`
- Stopping gradients
- How it's made: two foundational autodiff functions
- Composing VJPs, JVPs, and `vmap`
- Complex numbers and differentiation
- More advanced autodiff

JAX has a pretty general automatic differentiation system. In this document we'll go through a whole bunch of neat autodiff ideas that you can cherry-pick for your own work. The basics of `jax.grad` (`argnums`, differentiating with respect to containers, `value_and_grad`, checking derivatives against finite differences) are covered in the 101 docs (Transformations: grad and vmap), so here we start one level up: Hessian-vector products, full Jacobians, the `jvp`/`vjp` machinery underneath it all, and differentiation with complex numbers.

```
import jax.numpy as jnp
from jax import grad, jit, vmap
from jax import random

key = random.key(0)
```

Setup: a running example

As a running example, we'll use the same logistic regression model as the 101 docs:

latest

```
def sigmoid(x):
    return 0.5 * (jnp.tanh(x / 2) + 1)

# Outputs probability of a label being true.
def predict(W, b, inputs):
    return sigmoid(jnp.dot(inputs, W) + b)

# Build a toy dataset.
inputs = jnp.array([[0.52, 1.12,  0.77],
                    [0.88, -1.08, 0.15],
                    [0.52, 0.06, -1.30],
                    [0.74, -2.49, 1.39]])
targets = jnp.array([True, True, False, True])

# Training loss is the negative log-likelihood of the training examples.
def loss(W, b):
    preds = predict(W, b, inputs)
    label_probs = preds * targets + (1 - preds) * (1 - targets)
    return -jnp.sum(jnp.log(label_probs))

# Initialize random model coefficients
key, W_key, b_key = random.split(key, 3)
W = random.normal(W_key, (3,))
b = random.normal(b_key, ())
```

Jacobians and Hessians using `jacfwd` and `jacrev`

You can compute full Jacobian matrices using the `jacfwd` and `jacrev` functions:

```
from jax import jacfwd, jacrev

# Isolate the function from the weight matrix to the predictions
f = lambda W: predict(W, b, inputs)

J = jacfwd(f)(W)
print("jacfwd result, with shape", J.shape)
print(J)

J = jacrev(f)(W)
print("jacrev result, with shape", J.shape)
print(J)
```

```
jacfwd result, with shape (4, 3)
[[ 0.05069415  0.10918739  0.07506634]
 [ 0.14170025 -0.17390487  0.02415345]
 [ 0.12579198  0.01451446 -0.31447995]
 [ 0.00574409 -0.0193281   0.01078958]]
jacrev result, with shape (4, 3)
[[ 0.05069415  0.10918739  0.07506634]
 [ 0.14170025 -0.17390487  0.02415345]
 [ 0.12579198  0.01451446 -0.31447995]
 [ 0.00574409 -0.0193281   0.01078958]]
```

These two functions compute the same values (up to machine numerics), but differ in their implementation: `jacfwd` uses forward-mode automatic differentiation, which is more efficient for “tall” Jacobian matrices (more outputs than inputs), while `jacrev` uses reverse-mode, which is more efficient for “wide” Jacobian matrices (more inputs than outputs). For matrices that are near-square, `jacfwd` probably has an edge over `jacrev`.

You can also use `jacfwd` and `jacrev` with container types:

```
def predict_dict(params, inputs):
    return predict(params['W'], params['b'], inputs)

J_dict = jacrev(predict_dict)({'W': W, 'b': b}, inputs)
for k, v in J_dict.items():
    print("Jacobian from {} to logits is".format(k))
    print(v)
```

```
Jacobian from W to logits is
[[ 0.05069415  0.10918739  0.07506634]
 [ 0.14170025 -0.17390487  0.02415345]
 [ 0.12579198  0.01451446 -0.31447995]
 [ 0.00574409 -0.0193281   0.01078958]]
Jacobian from b to logits is
[0.09748875 0.16102302 0.24190766 0.00776229]
```

For more details on forward- and reverse-mode, as well as how to implement `jacfwd` and `jacrev` as efficiently as possible, read on!

Using a composition of two of these functions gives us a way to compute dense Hessian matrices:

```
def hessian(f):
    return jacfwd(jacrev(f))

H = hessian(f)(W)
print("hessian, with shape", H.shape)
print(H)
```

latest

```
hessian, with shape (4, 3, 3)
[[[ 0.02058932  0.04434624  0.03048804]
  [ 0.04434624  0.09551498  0.06566655]
  [ 0.03048804  0.06566655  0.04514575]]

 [[-0.0743913   0.09129842 -0.01268034]
  [ 0.09129842 -0.11204806  0.01556223]
  [-0.01268034  0.01556223 -0.00216142]]

 [[ 0.01176856  0.00135791 -0.0294214 ]
  [ 0.00135791  0.00015668 -0.00339478]
  [-0.0294214  -0.00339478  0.0735535 ]]

 [[-0.00418412  0.014079   -0.00785936]
  [ 0.014079   -0.04737393  0.02644569]
  [-0.00785936  0.02644569 -0.01476286]]]
```

This shape makes sense: if we start with a function $f : \mathbb{R}^n \to \mathbb{R}^m$, then at a point $x \in \mathbb{R}^n$ we expect to get the shapes

- $f(x) \in \mathbb{R}^m$, the value of f at x,
- $\partial f(x) \in \mathbb{R}^{m \times n}$, the Jacobian matrix at x,
- $\partial^2 f(x) \in \mathbb{R}^{m \times n \times n}$, the Hessian at x,

and so on.

To implement `hessian`, we could have used `jacfwd(jacrev(f))` or `jacrev(jacfwd(f))` or any other composition of the two. But forward-over-reverse is typically the most efficient. That's because in the inner Jacobian computation we're often differentiating a function with a wide Jacobian (maybe like a loss function $f : \mathbb{R}^n \to \mathbb{R}$), while in the outer Jacobian computation we're differentiating a function with a square Jacobian (since $\nabla f : \mathbb{R}^n \to \mathbb{R}^n$), which is where forward-mode wins out.

Stopping gradients

Sometimes you want autodiff to treat a value as a constant: use it as usual in the forward pass, but propagate no derivative through it. That's what `jax.lax.stop_gradient` does: it's the identity function, with a derivative that's identically zero.

One classic use is optimizing against a target built from the same parameters you're differentiating. In TD(0) reinforcement-learning updates, for example, the value-function target depends on the parameters, but a correct update requires treatin

latest

```
from jax import lax

def value_fn(theta, state):
  return jnp.dot(theta, state)

def td_loss(theta, s_prev, r, s):
  v_prev = value_fn(theta, s_prev)
  target = r + value_fn(theta, s)
  return -0.5 * (lax.stop_gradient(target) - v_prev) ** 2

theta = jnp.array([0.1, -0.1, 0.])
s_prev, r, s = jnp.array([1., 2., -1.]), 1., jnp.array([2., 1., 0.])
print(grad(td_loss)(theta, s_prev, r, s))
```

```
[ 1.2  2.4 -1.2]
```

Without the `stop_gradient`, the gradient would include a second term flowing through `target`, a different (and here, unwanted) algorithm.

Another classic is the *straight-through estimator*: apply a non-differentiable (or zero-derivative) function in the forward pass, but differentiate as if it were the identity. There's a well-known `stop_gradient` one-liner for it:

```
def straight_through_round(x):
  return x + lax.stop_gradient(jnp.round(x) - x)

print(straight_through_round(1.7))        # rounds like round
print(grad(straight_through_round)(1.7))  # differentiates like the identity
```

```
2.0
1.0
```

In the forward pass the `x`s cancel, leaving `round(x)`; under differentiation the `stop_gradient` term contributes zero, leaving the derivative of the identity. (For more principled ways to attach this kind of custom derivative behavior to functions or types, see Custom JVPs or VJPs with custom_jvp and custom_vjp and the straight-through example in Defining new JAX types with hijax.)

How it's made: two foundational autodiff functions

latest

Jacobian-Vector products (JVPs, aka forward-mode autodiff)

JAX includes efficient and general implementations of both forward- and reverse-mode automatic differentiation. The familiar `grad` function is built on reverse-mode, but to explain the difference between the two modes, and when each can be useful, we need a bit of math background.

JVPs in math

Mathematically, given a function $f : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian of f evaluated at an input point $x \in \mathbb{R}^n$, denoted $\partial f(x)$, is often thought of as a matrix:

$$\partial f(x) \in \mathbb{R}^{m \times n}.$$

But we can also think of $\partial f(x)$ as a linear map, which maps the tangent space of the domain of f at the point x (which is just another copy of $\mathbb{R}^n$) to the tangent space of the codomain of f at the point $f(x)$ (a copy of $\mathbb{R}^m$):

$$\partial f(x) : \mathbb{R}^n \to \mathbb{R}^m.$$

This map is called the pushforward map of f at x. The Jacobian matrix is just the matrix for this linear map in a standard basis.

If we don't commit to one specific input point x, then we can think of the function ∂f as first taking an input point and returning the Jacobian linear map at that input point:

$$\partial f : \mathbb{R}^n \to \mathbb{R}^n \to \mathbb{R}^m.$$

In particular, we can uncurry things so that given input point $x \in \mathbb{R}^n$ and a tangent vector $v \in \mathbb{R}^n$, we get back an output tangent vector in $\mathbb{R}^m$. We call that mapping, from (x, v) pairs to output tangent vectors, the *Jacobian-vector product*, and write it as

$$(x, v) \mapsto \partial f(x) v$$

JVPs in JAX code

Back in Python code, JAX's `jvp` function models this transformation. Given a Python function that evaluates f, JAX's `jvp` is a way to get a Python function $(x, v) \mapsto (f(x), \partial f(x) v)$.

latest

```
from jax import jvp

# Isolate the function from the weight matrix to the predictions
f = lambda W: predict(W, b, inputs)

key, subkey = random.split(key)
v = random.normal(subkey, W.shape)

# Push forward the vector `v` along `f` evaluated at `W`
y, u = jvp(f, (W,), (v,))
```

In terms of Haskell-like type signatures, we could write

```
jvp :: (a -> b) -> a -> T a -> (b, T b)
```

where we use `T a` to denote the type of the tangent space for `a`. In words, `jvp` takes as arguments a function of type `a -> b`, a value of type `a`, and a tangent vector value of type `T a`. It gives back a pair consisting of a value of type `b` and an output tangent vector of type `T b`.

The `jvp`-transformed function is evaluated much like the original function, but paired up with each primal value of type `a` it pushes along tangent values of type `T a`. For each primitive numerical operation that the original function would have applied, the `jvp`-transformed function executes a "JVP rule" for that primitive that both evaluates the primitive on the primals and applies the primitive's JVP at those primal values.

That evaluation strategy has some immediate implications about computational complexity: since we evaluate JVPs as we go, we don't need to store anything for later, and so the memory cost is independent of the depth of the computation. In addition, the FLOP cost of the `jvp`-transformed function is about 3x the cost of just evaluating the function (one unit of work for evaluating the original function, for example `sin(x)`; one unit for linearizing, like `cos(x)`; and one unit for applying the linearized function to a vector, like `cos_x * v`). Put another way, for a fixed primal point x, we can evaluate $v \mapsto \partial f(x) \cdot v$ for about the same marginal cost as evaluating f.

That memory complexity sounds pretty compelling! So why don't we see forward-mode very often in machine learning?

To answer that, first think about how you could use a JVP to build a full Jacobian matrix. If we apply a JVP to a one-hot tangent vector, it reveals one column of the Jacobian matrix, corresponding to the nonzero entry we fed in. So we can build a full Jac
latest
a time, and to get each column costs about the same as one function evaluation. That will be efficient for functions with "tall" Jacobians, but inefficient for "wide" Jacobians.

If you're doing gradient-based optimization in machine learning, you probably want to minimize a loss function from parameters in $\mathbb{R}^n$ to a scalar loss value in $\mathbb{R}$. That means the Jacobian of this function is a very wide matrix: $\partial f(x) \in \mathbb{R}^{1\times n}$, which we often identify with the gradient vector $\nabla f(x) \in \mathbb{R}^n$. Building that matrix one column at a time, with each call taking a similar number of FLOPs to evaluate the original function, sure seems inefficient! In particular, for training neural networks, where f is a training loss function and n can be in the millions or billions, this approach just won't scale.

To do better for functions like this, we just need to use reverse-mode.

Vector-Jacobian products (VJPs, aka reverse-mode autodiff)

Where forward-mode gives us back a function for evaluating Jacobian-vector products, which we can then use to build Jacobian matrices one column at a time, reverse-mode is a way to get back a function for evaluating vector-Jacobian products (equivalently Jacobian-transpose-vector products), which we can use to build Jacobian matrices one row at a time.

VJPs in math

Let's again consider a function $f : \mathbb{R}^n \to \mathbb{R}^m$. Starting from our notation for JVPs, the notation for VJPs is pretty simple:

$$(x, v) \mapsto v\partial f(x),$$

where v is an element of the cotangent space of the codomain of f at $f(x)$ (isomorphic to another copy of $\mathbb{R}^m$). When being rigorous, we should think of v as a linear map $v : \mathbb{R}^m \to \mathbb{R}$, and when we write $v\partial f(x)$ we mean function composition $v \circ \partial f(x)$, where the types work out because $\partial f(x) : \mathbb{R}^n \to \mathbb{R}^m$. But in the common case we can identify v with a vector in $\mathbb{R}^m$ and use the two almost interchangeably, just like we might sometimes flip between "column vectors" and "row vectors" without much comment.

With that identification, we can alternatively think of the linear part of a VJP as the transpose (or adjoint) of the linear part of a JVP:

$$(x, v) \mapsto \partial f(x)^\mathsf{T} v.$$

For a given point x, we can write the signature as

$$\partial f(x)^\mathsf{T} : \mathbb{R}^m \to \mathbb{R}^n.$$

latest

The corresponding map on cotangent spaces is often called the pullback of f at x. The key for our purposes is that it goes from something that looks like the output of f to something that looks like the input of f, just like we might expect from a transposed linear function.

VJPs in JAX code

Switching from math back to Python, the JAX function `vjp` can take a Python function for evaluating f and give us back a Python function for evaluating the VJP $(x, v) \mapsto (f(x), v^\mathsf{T} \partial f(x))$.

```
from jax import vjp

# Isolate the function from the weight matrix to the predictions
f = lambda W: predict(W, b, inputs)

y, vjp_fun = vjp(f, W)

key, subkey = random.split(key)
u = random.normal(subkey, y.shape)

# Pull back the covector `u` along `f` evaluated at `W`
v = vjp_fun(u)
```

In terms of Haskell-like type signatures, we could write

```
vjp :: (a -> b) -> a -> (b, CT b -> CT a)
```

where we use `CT a` to denote the type for the cotangent space for `a`. In words, `vjp` takes as arguments a function of type `a -> b` and a point of type `a`, and gives back a pair consisting of a value of type `b` and a linear map of type `CT b -> CT a`.

This is great because it lets us build Jacobian matrices one row at a time, and the FLOP cost for evaluating $(x, v) \mapsto (f(x), v^\mathsf{T} \partial f(x))$ is only about three times the cost of evaluating f. In particular, if we want the gradient of a function $f : \mathbb{R}^n \to \mathbb{R}$, we can do it in just one call. That's how `grad` is efficient for gradient-based optimization, even for objectives like neural network training loss functions on millions or billions of parameters.

There's a cost, though: while the FLOPs are friendly, memory scales with the depth of the computation. Also, the implementation is traditionally more complex than that of forward-mode, though JAX has a trick up its sleeve: as we'll see next, it builds reverse-mode out of forward-mode.

latest

For more on how reverse-mode works, see [this tutorial video from the Deep Learning Summer School in 2017](#).

`jax.linearize`: one forward pass, many JVPs

Every call to `jvp(f, (x,), (v,))` redoes the forward pass: it evaluates $f(x)$ alongside $\partial f(x)v$. If you want Jacobian-vector products against many different vectors v at the *same* point x, that's wasteful: the work that depends only on x could be done once. That's what `jax.linearize` provides:

```
from jax import linearize

f = lambda W: predict(W, b, inputs)

y, f_jvp = linearize(f, W)
print(y)
```

```
[0.10947311 0.79829013 0.41004258 0.99217653]
```

In terms of Haskell-like type signatures,

```
linearize :: (a -> b) -> a -> (b, T a -> T b)
```

Like `vjp`, `linearize` returns the primal output paired with a function; but here the function is the *pushforward* $v \mapsto \partial f(x)v$, with the forward pass already done and the intermediates it needs saved. Each call to `f_jvp` costs roughly the linear part of the work alone:

```
key, k1, k2 = random.split(key, 3)
v1, v2 = random.normal(k1, W.shape), random.normal(k2, W.shape)
print(f_jvp(v1))
print(f_jvp(v2))
```

```
[ 0.01550586 -0.29472184 -0.8186045   0.00393129]
[ 0.11144301 -0.28251454 -0.0085168  -0.02786912]
```

Besides the efficiency, `linearize` gives us a way to think about reverse-mode. The function `f_jvp` is guaranteed to be *linear*, and a linear function can be transposed (JAX exposes that operation as `jax.linear_transpose`). Reverse-mode is the comp
linearize the function at x, then transpose the resulting linear map:

latest

```
from jax import linear_transpose, vjp

f_vjp = linear_transpose(f_jvp, W)      # transpose of the linearization...

y2, f_vjp_ref = vjp(f, W)               # ...is exactly what vjp computes

u = jnp.ones_like(y)
print(jnp.allclose(f_vjp(u)[0], f_vjp_ref(u)[0]))
```

```
True
```

This “linearize, then transpose” decomposition is how JAX implements `vjp` and `grad` internally, and the rest of this documentation section uses its vocabulary: when we say one operation “transposes to” another (as in Autodiff and sharding and Custom derivative rules with hijax primitives), we mean this transposition of the linearized computation.

Vector-valued gradients with VJPs

If you’re interested in taking vector-valued gradients (like `tf.gradients`):

```
from jax import vjp

def vgrad(f, x):
  y, vjp_fn = vjp(f, x)
  return vjp_fn(jnp.ones(y.shape))[0]

print(vgrad(lambda x: 3*x**2, jnp.ones((2, 2))))
```

```
[[6. 6.]
 [6. 6.]]
```

Hessian-vector products using both forward- and reverse-mode

A Hessian-vector product function can be useful in a truncated Newton Conjugate-Gradient algorithm for minimizing smooth convex functions, or for studying the curvature of neural network training objectives (e.g. 1, 2, 3, 4). The trick is to evaluate products $v \mapsto \partial^2 f(x) \cdot v$ *without instantiating the Hessian matrix*: for a scalar loss on millions or

latest

the full $n \times n$ Hessian is impossible to store, but a Hessian-vector proc

constant multiple of evaluating f.

The most efficient recipe uses forward-over-reverse composition.

Mathematically, given a function $f : \mathbb{R}^n \to \mathbb{R}$ to differentiate, a point $x \in \mathbb{R}^n$ at which to linearize the function, and a vector $v \in \mathbb{R}^n$, the Hessian-vector product function we want is

$$(x, v) \mapsto \partial^2 f(x) v$$

Consider the helper function $g : \mathbb{R}^n \to \mathbb{R}^n$ defined to be the derivative (or gradient) of f, namely $g(x) = \partial f(x)$. All we need is its JVP, since that will give us

$$(x, v) \mapsto \partial g(x) v = \partial^2 f(x) v.$$

We can translate that almost directly into code:

```
from jax import jvp, grad

# forward-over-reverse
def hvp(f, primals, tangents):
  return jvp(grad(f), primals, tangents)[1]
```

As a bonus, since we didn't have to call `jnp.dot` directly, this `hvp` function works with arrays of any shape and with arbitrary container types (like vectors stored as nested lists/dicts/tuples), and doesn't even have a dependence on `jax.numpy`.

Here's an example of how to use it:

```
def f(X):
  return jnp.sum(jnp.tanh(X)**2)

key, subkey1, subkey2 = random.split(key, 3)
X = random.normal(subkey1, (30, 40))
V = random.normal(subkey2, (30, 40))

ans1 = hvp(f, (X,), (V,))
ans2 = jnp.tensordot(hessian(f)(X), V, 2)

print(jnp.allclose(ans1, ans2, 1e-4, 1e-4))
```

```
True
```

Another way you might consider writing this is using reverse-over-forward:

latest

```
# reverse-over-forward
def hvp_revfwd(f, primals, tangents):
  g = lambda primals: jvp(f, primals, tangents)[1]
  return grad(g)(primals)
```

That's not quite as good, though, because forward-mode has less overhead than reverse-mode, and since the outer differentiation operator here has to differentiate a larger computation than the inner one, keeping forward-mode on the outside works best. For comparison, here's reverse-over-reverse too, with all three timed against materializing the full Hessian:

```
# reverse-over-reverse, only works for single arguments
def hvp_revrev(f, primals, tangents):
  x, = primals
  v, = tangents
  return grad(lambda x: jnp.vdot(grad(f)(x), v))(x)


print("Forward over reverse")
%timeit -n10 -r3 hvp(f, (X,), (V,))
print("Reverse over forward")
%timeit -n10 -r3 hvp_revfwd(f, (X,), (V,))
print("Reverse over reverse")
%timeit -n10 -r3 hvp_revrev(f, (X,), (V,))

print("Naive full Hessian materialization")
%timeit -n10 -r3 jnp.tensordot(hessian(f)(X), V, 2)
```

```
Forward over reverse
3.67 ms ± 121 µs per loop (mean ± std. dev. of 3 runs, 10 loops each)
Reverse over forward
10.3 ms ± 6.97 ms per loop (mean ± std. dev. of 3 runs, 10 loops each)
Reverse over reverse
The slowest run took 4.48 times longer than the fastest. This could mean that
15.2 ms ± 11.5 ms per loop (mean ± std. dev. of 3 runs, 10 loops each)
Naive full Hessian materialization
39.4 ms ± 1.1 ms per loop (mean ± std. dev. of 3 runs, 10 loops each)
```

Composing VJPs, JVPs, and `vmap`

Jacobian-Matrix and Matrix-Jacobian products

latest

Now that we have `jvp` and `vjp` transformations that give us functions pull back one vector at a time, we can use JAX's `vmap` transformation (Transformations:

grad and vmap) to push and pull entire bases at once. In particular, we can use that to write fast matrix-Jacobian and Jacobian-matrix products.

```
# Isolate the function from the weight matrix to the predictions
f = lambda W: predict(W, b, inputs)

# Pull back the covectors `m_i` along `f`, evaluated at `W`, for all `i`.
# First, use a list comprehension to loop over rows in the matrix M.
def loop_mjp(f, x, M):
    y, vjp_fun = vjp(f, x)
    return jnp.vstack([jnp.asarray(vjp_fun(mi)) for mi in M])

# Now, use vmap to build a computation that does a single fast matrix-matrix
# multiply, rather than an outer loop over vector-matrix multiplies.
def vmap_mjp(f, x, M):
    y, vjp_fun = vjp(f, x)
    outs, = vmap(vjp_fun)(M)
    return outs

key = random.key(0)
num_covecs = 128
U = random.normal(key, (num_covecs,) + y.shape)

loop_vs = loop_mjp(f, W, M=U)
print('Non-vmapped Matrix-Jacobian product')
%timeit -n10 -r3 loop_mjp(f, W, M=U)

print('\nVmapped Matrix-Jacobian product')
vmap_vs = vmap_mjp(f, W, M=U)
%timeit -n10 -r3 vmap_mjp(f, W, M=U)

assert jnp.allclose(loop_vs, vmap_vs), 'Vmap and non-vmapped Matrix-Jacobian
```

```
Non-vmapped Matrix-Jacobian product
116 ms ± 180 µs per loop (mean ± std. dev. of 3 runs, 10 loops each)

Vmapped Matrix-Jacobian product
5.02 ms ± 73.6 µs per loop (mean ± std. dev. of 3 runs, 10 loops each)
```

latest

```
def loop_jmp(f, W, M):
    # jvp immediately returns the primal and tangent values as a tuple,
    # so we'll compute and select the tangents in a list comprehension
    return jnp.vstack([jvp(f, (W,), (mi,))[1] for mi in M])

def vmap_jmp(f, W, M):
    _jvp = lambda s: jvp(f, (W,), (s,))[1]
    return vmap(_jvp)(M)

num_vecs = 128
S = random.normal(key, (num_vecs,) + W.shape)

loop_vs = loop_jmp(f, W, M=S)
print('Non-vmapped Jacobian-Matrix product')
%timeit -n10 -r3 loop_jmp(f, W, M=S)
vmap_vs = vmap_jmp(f, W, M=S)
print('\nVmapped Jacobian-Matrix product')
%timeit -n10 -r3 vmap_jmp(f, W, M=S)

assert jnp.allclose(loop_vs, vmap_vs), 'Vmap and non-vmapped Jacobian-Matrix
```

```
Non-vmapped Jacobian-Matrix product
130 ms ± 98.1 µs per loop (mean ± std. dev. of 3 runs, 10 loops each)

Vmapped Jacobian-Matrix product
1.84 ms ± 32.2 µs per loop (mean ± std. dev. of 3 runs, 10 loops each)
```

The implementation of `jacfwd` and `jacrev`

Now that we've seen fast Jacobian-matrix and matrix-Jacobian products, it's not hard to guess how to write `jacfwd` and `jacrev`. We just use the same technique to push forward or pull back an entire standard basis (isomorphic to an identity matrix) at once.

```
from jax import jacrev as builtin_jacrev

def our_jacrev(f):
    def jacfun(x):
        y, vjp_fun = vjp(f, x)
        # Use vmap to do a matrix-Jacobian product.
        # Here, the matrix is the Euclidean basis, so we get all
        # entries in the Jacobian at once.
        J, = vmap(vjp_fun, in_axes=0)(jnp.eye(len(y)))
        return J
    return jacfun

assert jnp.allclose(builtin_jacrev(f)(W), our_jacrev(f)(W)), 'Incorrect reve
```

latest

```
from jax import jacfwd as builtin_jacfwd

def our_jacfwd(f):
    def jacfun(x):
        _jvp = lambda s: jvp(f, (x,), (s,))[1]
        Jt = vmap(_jvp, in_axes=1)(jnp.eye(len(x)))
        return jnp.transpose(Jt)
    return jacfun

assert jnp.allclose(builtin_jacfwd(f)(W), our_jacfwd(f)(W)), 'Incorrect forw
```

Interestingly, Autograd couldn't do this. Our implementation of reverse-mode `jacobian` in Autograd had to pull back one vector at a time with an outer-loop `map`. Pushing one vector at a time through the computation is much less efficient than batching it all together with `vmap`.

Another thing that Autograd couldn't do is `jit`. No matter how much Python dynamism you use in the function being differentiated, we can always use `jit` on the linear part of the computation. For example:

```
def f(x):
    try:
        if x < 3:
            return 2 * x ** 3
        else:
            raise ValueError
    except ValueError:
        return jnp.pi * x

y, f_vjp = vjp(f, 4.)
print(jit(f_vjp)(1.))
```

```
(Array(3.1415927, dtype=float32, weak_type=True),)
```

In fact, the callable returned by `jax.vjp` is a first-class value in its own right, a pytree whose leaves are the saved residuals. You can use it to split the forward and backward passes into separately compiled functions, schedule them yourself, and control what gets saved. That's the subject of First-class VJPs.

Complex numbers and differentiation

latest

In JAX, differentiation of complex-valued functions is defined in terms o derivatives. Jacobian-vector products (JVPs) and vector-Jacobian products (VJPs) operate on real-linear maps without requiring holomorphy. The only convention choice is how

covectors are represented as complex numbers: JAX identifies them with vectors via a bilinear pairing rather than a sesquilinear one, which shows up in `grad` as a complex conjugate relative to the gradient vector.

The unambiguous part: JVPs and VJPs

Under the identification $\mathbb{C} \cong \mathbb{R}^2$, any function $f : \mathbb{C} \to \mathbb{C}$ corresponds to a real function $F : \mathbb{R}^2 \to \mathbb{R}^2$ defined by

$$f(x + yi) = u(x, y) + v(x, y)i \quad \leftrightarrow \quad F(x, y) = (u(x, y), v(x, y)).$$

The derivative of F at a point is the real 2×2 Jacobian matrix

$$J = \begin{bmatrix} \partial_0 u & \partial_1 u \\ \partial_0 v & \partial_1 v \end{bmatrix}.$$

The **JVP** of f is the pushforward defined by this real linear map applied to tangent vectors, with complex numbers serving as representations for pairs of reals: for a tangent $t = t_1 + t_2 i$, the output tangent is the complex representation of $J(t_1, t_2)$. This definition applies to all differentiable functions regardless of holomorphy:

```
def u(x, y): return x**2 + jnp.sin(y)
def v(x, y): return x * y

def fun(z):  # not holomorphic!
  x, y = jnp.real(z), jnp.imag(z)
  return u(x, y) + v(x, y) * 1j

z = 1.5 + 0.5j
x, y = jnp.real(z), jnp.imag(z)
J = jnp.array([[grad(u, 0)(x, y), grad(u, 1)(x, y)],
               [grad(v, 0)(x, y), grad(v, 1)(x, y)]])

t = 0.7 - 0.3j
_, t_out = jvp(fun, (z,), (t,))
t_pair = J @ jnp.array([jnp.real(t), jnp.imag(t)])
print(jnp.allclose(t_out, t_pair[0] + t_pair[1] * 1j))
```

```
True
```

When f is holomorphic, the Cauchy–Riemann equations imply that J is a scaled rotation corresponding to complex multiplication by $f'(z)$, and the JVP reduces to $t \mapsto f'(z)\, t$.

latest

The **VJP** is the pullback (dual map) of the derivative. The derivative's dual map sends an output linear functional φ to an input linear functional by composition, $\varphi \mapsto \varphi \circ \partial F(x)$.

Because f is in general only $\mathbb{R}$-differentiable, its tangent and cotangent spaces are vector spaces over $\mathbb{R}$, and cotangents are $\mathbb{R}$-linear functionals into $\mathbb{R}$, not complex numbers.

JAX's `vjp` nonetheless returns cotangents with the same type as the primal values, so covectors must be *represented* as complex numbers, and that requires identifying each functional with a number. A nondegenerate real-valued pairing $\langle \cdot, \cdot \rangle$ is exactly such an identification: it matches the number w with the functional $\langle w, \cdot \rangle$. Relative to chosen pairings on the domain and codomain, the transpose A^T of an $\mathbb{R}$-linear map A is characterized by

$$\langle w, At \rangle = \langle A^\mathsf{T} w, t \rangle \quad \text{for all } t, w.$$

On $\mathbb{C} \cong \mathbb{R}^2$, there are two standard choices of real-valued pairing:

- the **bilinear** pairing $\langle w, t \rangle = \mathrm{Re}(wt) = w_1 t_1 - w_2 t_2$;
- the **sesquilinear** pairing $\langle w, t \rangle = \mathrm{Re}(\bar{w} t) = w_1 t_1 + w_2 t_2$, which is the standard Euclidean inner product on $\mathbb{R}^2$.

These pairings differ by a conjugation in the first argument, so the transposes they induce differ by an elementwise conjugation. **JAX adopts the bilinear pairing**.

To unpack the functional from the number: a cotangent $w = w_1 + w_2 i$ returned by `vjp` encodes the $\mathbb{R}$-linear functional $t \mapsto \mathrm{Re}(wt)$, which acts on the real pair (t_1, t_2) as $w_1 t_1 - w_2 t_2$. Its components against the standard dual basis are therefore $(w_1, -w_2)$, the components of $\bar{w}$. This is the conjugation that resurfaces in `grad` below.

Under this convention, `vjp` is characterized by

$$\mathrm{Re}(w \cdot \mathtt{jvp}(t)) \;=\; \mathrm{Re}(\mathtt{vjp}(w) \cdot t) \qquad \text{for all } t, w,$$

using standard complex products without explicit conjugation:

```
w = -0.2 + 1.1j

_, fun_vjp = vjp(fun, z)
w_out, = fun_vjp(w)

print(jnp.allclose(jnp.real(w * t_out), jnp.real(w_out * t)))       # True
print(jnp.allclose(jnp.real(jnp.conj(w) * t_out),
                   jnp.real(jnp.conj(w_out) * t)))                  # False!
```

```
True
False
```

latest

Furthermore, if f is holomorphic, both the JVP and the VJP simplify to complex multiplication:

$$\texttt{jvp}(t) = f'(z)\,t, \qquad \texttt{vjp}(w) = f'(z)\,w.$$

```
_, t_out = jvp(jnp.sin, (z,), (t,))
_, sin_vjp = vjp(jnp.sin, z)
w_out, = sin_vjp(w)
print(jnp.allclose(t_out, jnp.cos(z) * t))
print(jnp.allclose(w_out, jnp.cos(z) * w))
```

```
True
True
```

Under the sesquilinear pairing, the VJP of a holomorphic function would evaluate to $w \mapsto \overline{f'(z)}\,w$.

A general $\mathbb{R}$-differentiable map $\mathbb{C} \to \mathbb{C}$ has four real degrees of freedom in its derivative. A single JVP or VJP computes a two-dimensional projection; two evaluations on linearly independent tangents (such as 1 and i) recover the full Jacobian. For functions with a real domain or codomain, a single evaluation suffices: one `jvp` determines the derivative of an $\mathbb{R} \to \mathbb{C}$ function, and one `vjp` (or `grad`) determines the derivative of a $\mathbb{C} \to \mathbb{R}$ function. When in doubt about what a complex derivative means, use `jvp` and `vjp` directly: they are always well-defined, for any function.

`grad` at complex inputs

For a scalar function $f : \mathbb{C} \to \mathbb{R}$ with $f(x + yi) = u(x, y)$, JAX defines `grad(f)(z)` as `vjp(f, z)[1](1.0)[0]`. Applying the bilinear transpose formula gives

$$\texttt{grad}(f)(z) = \partial_0 u(x, y) - \partial_1 u(x, y)\,i.$$

This is the complex conjugate of the gradient vector $(\partial_0 u, \partial_1 u)$ in $\mathbb{R}^2$. Under the bilinear convention, directional derivatives are given directly by the real part of the product:

$$\lim_{\epsilon \to 0} \tfrac{1}{\epsilon}(f(z + \epsilon t) - f(z)) = \mathrm{Re}(\texttt{grad}(f)(z) \cdot t).$$

Consequently, the direction of steepest ascent in the complex plane is $\overline{\texttt{grad}(f)(z)}$, and gradient descent updates take the form $z \leftarrow z - \eta\,\overline{\texttt{grad}(f)(z)}$. Updating along the unconjugated gradient negates the imaginary component of the descent step:

latest

```
def f(z):
  x, y = jnp.real(z), jnp.imag(z)
  return x**2 + y**2      # |z|^2, minimized at z = 0

print(grad(f)(3. + 4j))   # 6 - 8j: conjugate of the steepest-ascent 6 + 8j
```

```
(6-8j)
```

```
z = 3. + 4j
for _ in range(100):
  z = z - 0.05 * jnp.conj(grad(f)(z))   # with the conjugate: descends
print(f(z))

z = 3. + 4j
for _ in range(100):
  z = z - 0.05 * grad(f)(z)             # without: the imaginary part grows!
print(f(z))
```

```
1.7637703e-08
3.0384858e+09
```

Under JAX's convention:

1. Optimization of a real-valued loss over complex parameters requires stepping along the conjugate of `grad`, $\overline{\mathbf{grad}(f)(z)}$. Optimizer libraries written with real parameters in mind do not apply this conjugation, so audit any optimizer you use on complex parameters.
2. First-order Taylor approximations and directional derivatives use the unconjugated product: $f(z+t) \approx f(z) + \mathrm{Re}(\mathbf{grad}(f)(z) \cdot t)$.

Why the bilinear convention?

The two pairings differ by an involution, so they carry the same information; choosing between them decides only where conjugations appear. The choice is a tradeoff.

What the bilinear pairing provides:

1. **Holomorphic functions require no conjugation.** Both the JVP and the VJP are plain multiplication by $f'(z)$, and `grad(f, holomorphic=True)` returns than its conjugate.

latest

2. **Plain-product identities.** The characterization identity above and the first-order Taylor expansion $f(z+t) \approx f(z) + \mathrm{Re}(\mathtt{grad}(f)(z) \cdot t)$ use ordinary complex products, with no conjugations.
3. **Derivative rules work unchanged from their real implementations.** Since $\mathrm{Re}((cw)t) = \mathrm{Re}(w(ct))$, multiplication by c transposes to multiplication by c. So for a holomorphic primitive, the VJP is the regular complex derivative multiplied by the incoming cotangent w, the same expression as in the real case, and most simple math primitives don't need their derivative rules changed from their real implementations. (For example, the VJP rule for `sin` is $w \mapsto w\cos(z)$ in the real and complex cases alike.) This holds whether VJP rules are written directly, as in Autograd, or derived by transposing linearized computations, as in JAX; under the sesquilinear convention, every such rule would need a conjugation added.

What the sesquilinear pairing would provide instead:

1. **`grad` is steepest ascent directly.** The sesquilinear pairing is the Euclidean inner product on $\mathbb{R}^2$, so its identification of covectors with vectors is the Riesz representation: `grad` would return the gradient vector itself, and optimizers written for real parameters would step correctly with no conjugation. This is the convention PyTorch and TensorFlow adopt (see the Wirtinger discussion below).
2. **Familiar metric duality.** Transposes relative to it are the conjugate transposes of ordinary linear algebra, and $\langle w, w\rangle = |w|^2 \geq 0$.

In summary: the bilinear convention puts the conjugation into optimizer steps, while the sesquilinear convention puts it into holomorphic derivatives and into every transpose rule. The former is algebraic duality (identifying $\mathbb{C}$ with its complex-linear dual), the latter metric duality (Riesz representation).

Relation to Wirtinger derivatives

Wirtinger derivatives provide an alternative coordinate representation of the real Jacobian:

$$\frac{\partial}{\partial z} = \frac{1}{2}\left(\frac{\partial}{\partial x} - i\frac{\partial}{\partial y}\right), \qquad \frac{\partial}{\partial \bar{z}} = \frac{1}{2}\left(\frac{\partial}{\partial x} + i\frac{\partial}{\partial y}\right).$$

The four real partial derivatives of a function $f : \mathbb{C} \to \mathbb{C}$ are parameterized by the two complex quantities $\partial f/\partial z$ and $\partial f/\partial \bar{z}$, and the JVP is expressed as

$$\mathtt{jvp}(t) = \frac{\partial f}{\partial z}\, t + \frac{\partial f}{\partial \bar{z}}\, \bar{t}.$$

latest

```
z = 1.5 + 0.5j                # the point where we computed J above

fx = J[0, 0] + J[1, 0] * 1j   # df/dx = du/dx + i dv/dx
fy = J[0, 1] + J[1, 1] * 1j   # df/dy = du/dy + i dv/dy
dfdz    = 0.5 * (fx - 1j * fy)
dfdzbar = 0.5 * (fx + 1j * fy)

_, t_out = jvp(fun, (z,), (t,))
print(jnp.allclose(t_out, dfdz * t + dfdzbar * jnp.conj(t)))
```

```
True
```

A function is holomorphic if and only if $\partial f/\partial \bar{z} = 0$ (the Cauchy–Riemann equations), in which case $\partial f/\partial z = f'(z)$. For a real-valued function f, JAX's `grad` computes $\mathbf{grad}(f)(z) = 2\,\partial f/\partial z$, whereas the steepest-ascent vector is $2\,\partial f/\partial \bar{z} = \overline{\mathbf{grad}(f)(z)}$. Other frameworks, including PyTorch and TensorFlow, define the gradient of a real-valued loss L as the steepest-ascent vector $2\,\partial L/\partial \bar{z}$, incorporating the conjugation into the returned derivative. Both approaches represent the same underlying real derivative under different identification conventions.

Holomorphic functions and `grad(f, holomorphic=True)`

For a function $f : \mathbb{C} \to \mathbb{C}$ with a complex output, `grad` raises an error because the four real derivative components cannot be uniquely represented by a single complex scalar. In general, `jax.vjp` or `jax.jvp` should be used directly.

When f is holomorphic, the derivative is completely characterized by the single complex number $f'(z)$, as the Cauchy–Riemann equations restrict the Jacobian to a scaled rotation. Setting `holomorphic=True` indicates that this condition holds, causing `grad` to evaluate the VJP at cotangent 1.0 and return $f'(z)$:

```
print(grad(jnp.sin, holomorphic=True)(3. + 4j))
print(jnp.cos(3. + 4j))
```

```
(-27.034945-3.8511531j)
(-27.034945-3.8511531j)
```

The `holomorphic=True` parameter disables the error check for comple

latest

verifying holomorphy. If applied to a non-holomorphic function, it returns the derivative of the

real component of the output, discarding the imaginary component:

```
def f(z):
  return jnp.conjugate(z)   # not holomorphic!

grad(f, holomorphic=True)(3. + 4j)
```

```
Array(1.-0.j, dtype=complex64, weak_type=True)
```

Complex numbers are supported across JAX transformations and linear algebra operations, including matrix factorizations. Here's a real-valued loss computed through the Cholesky factorization of a complex Hermitian matrix. Cholesky isn't holomorphic (it involves conjugation), so this is a job for plain `grad`, and we can check the result against a finite difference using the directional-derivative identity from above, summed over matrix entries:

```
A = jnp.array([[5.,    2.+3j,    5j],
               [2.-3j,   7.,  1.+7j],
               [-5j,  1.-7j,    12.]])

def f(X):
  L = jnp.linalg.cholesky(X)
  return jnp.sum(jnp.abs(L - jnp.sin(L))**2)   # real-valued

G = grad(f)(A)

T = jnp.array([[1., 1j, 0.], [-1j, 0., 2.], [0., 2., -1.]])  # a Hermitian d
eps = 1e-3
print((f(A + eps * T) - f(A - eps * T)) / (2 * eps))  # finite difference
print(jnp.real(jnp.sum(G * T)))                       # Re(grad(f)(A) · T)
```

```
3.5705564
3.5761378
```

More advanced autodiff

We worked through some easy, and then progressively more complicated, applications of automatic differentiation in JAX. We hope you now feel that taking derivatives in JAX is easy and powerful. The rest of this documentation section goes deeper:

- First-class VJPs — the VJP object as a pytree: splitting the forward passes into separately compiled functions, run on your own schedu

latest

- Autodiff and sharding — how autodiff interacts with sharding: the same cotangent-type reasoning as this page, extended to distributed arrays.
- Custom derivative rules with hijax primitives — defining your own derivative rules with hijax primitives (including efficient derivatives at fixed points), and Custom JVPs or VJPs with custom_jvp and custom_vjp for the classic decorator APIs.
- Autodiff with refs — how autodiff interacts with mutable arrays: plumbing values out of backward passes and accumulating gradients in place.
- Gradient checkpointing with jax.checkpoint (jax.remat) — controlling what autodiff saves versus recomputes, to trade memory for FLOPs.
- Defining new JAX types with hijax — differentiating with respect to entirely new types.